

Лабораторная работа №5

Анализ последовательных данных с помощью рекуррентных нейронных сетей

До сих пор мы имели дело со статическими данными. Однако искусственные нейронные сети могут пригодиться и при построении моделей последовательных данных. В частности, рекуррентные нейронные сети отлично справляются с моделированием таких данных. Пожалуй, именно последовательные данные в виде временных рядов встречаются чаще всего в нашем мире.

Работая с временными рядами данных, мы не можем просто использовать универсальные модели обучения. Для создания надежных моделей необходимо учитывать временные зависимости между данными. Давайте рассмотрим, как это делается.

Создайте новый файл Python и импортируйте следующие файлы.

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

Определим функцию, генерирующую синусоидальные сигналы. Начнем с определения четырех синусоидальных волн.

```
def get_data(num_points):
    # Создание синусоидальных волн
    wave_1 = 0.5 * np.sin(np.arange(0, num_points))
    wave_2 = 3.6 * np.sin(np.arange(0, num_points))
    wave_3 = 1.1 * np.sin(np.arange(0, num_points))
    wave_4 = 4.7 * np.sin(np.arange(0, num_points))
```

Создадим переменные амплитуды синусоидальных сигналов.

```
# Создание переменных амплитуд
amp_1 = np.ones(num_points)
amp_2 = 2.1 + np.zeros(num_points)
amp_3 = 3.2 * np.ones(num_points)
amp_4 = 0.8 + np.zeros(num_points)
```

Создадим суммарный волновой сигнал.

```
wave = np.array([wave_1, wave_2, wave_3,
                  wave_4]).reshape(num_points * 4, 1)
amp = np.array([amp_1, amp_2, amp_3,
                 amp_4]).reshape(num_points * 4, 1)

return wave, amp
```

Определим функцию, визуализирующую выходной сигнал нейронной сети.

```
# Визуализация выходного сигнала
def visualize_output(nn, num_points_test):
    wave, amp = get_data(num_points_test)
    output = nn.sim(wave)
    plt.plot(amp.reshape(num_points_test * 4))
    plt.plot(output.reshape(num_points_test * 4))
```

Определим основную функцию и создадим волновой сигнал.

```
if __name__ == '__main__':
    # Создание выборочных данных
    num_points = 40
    wave, amp = get_data(num_points)
```

Создадим рекуррентную нейронную сеть с двумя слоями.

```
# Создание рекуррентной нейронной сети с двумя слоями
nn = nl.net.newelm([-2, 2], [10, 1], [nl.trans.TanSig(),
                                     nl.trans.PureLin()])
```

Зададим функции инициализации для каждого слоя.

```
# Задание функций инициализации для каждого слоя
nn.layers[0].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
nn.layers[1].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
nn.init()
```

Обучим нейронную сеть.

```
# Обучение рекуррентной нейронной сети
error_progress = nn.train(wave, amp, epochs=1200, show=100,
                           goal=0.01)
```

Прогоним данные через нейронную сеть.

```
# Прогонка тренировочных данных через сеть
output = nn.sim(wave)
```

Построим график результатов.

```
# Построение графика результатов
plt.subplot(211)
plt.plot(error_progress)
plt.xlabel('Количество эпох')
plt.ylabel('Ошибка (MSE)')

plt.subplot(212)
plt.plot(amp.reshape(num_points * 4))
plt.plot(output.reshape(num_points * 4))
plt.legend(['Факт', 'Прогноз'])
```

Проверим работу нейронной сети, используя неизвестные тестовые данные.

```
# Тестирование нейронной сети на неизвестных данных
plt.figure()

plt.subplot(211)
visualize_output(nn, 82)
plt.xlim([0, 300])

plt.subplot(212)
visualize_output(nn, 49)
plt.xlim([0, 300])

plt.show()
```

В процессе выполнения этого кода на экране отобразятся два графика. В верхней части первого снимка экрана представлено продвижение процесса обучения, а в нижней — предсказанный результат, наложенный на суммарный входной сигнал (рис. 14.12).

В верхней части второго снимка экрана отображается результат имитации волнового сигнала нейронной сетью для увеличенной длины сигнала, а в нижней — для уменьшенной длины (рис. 14.13).

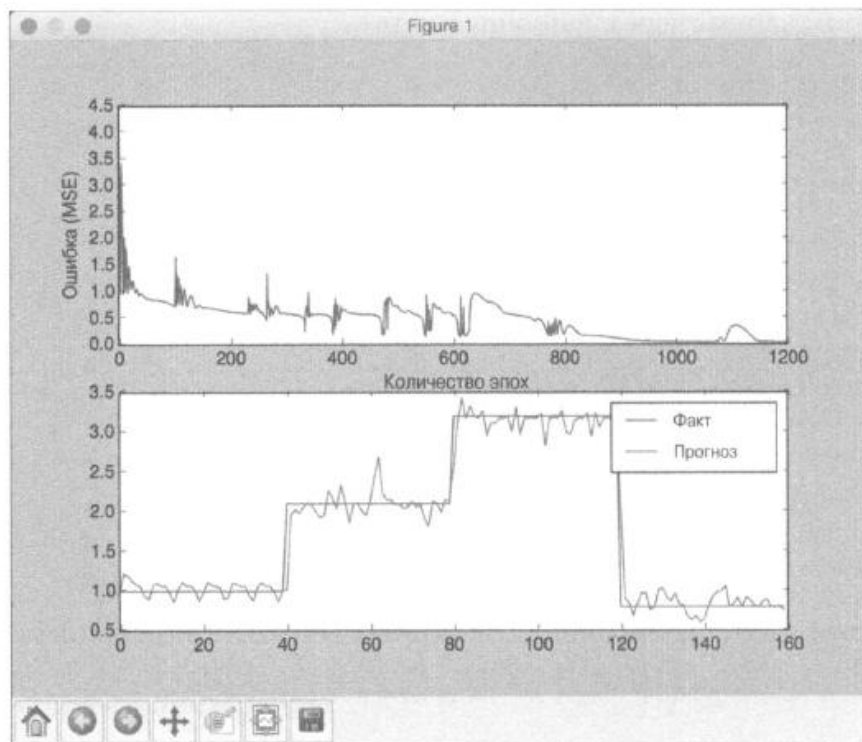


Рис. 14.12

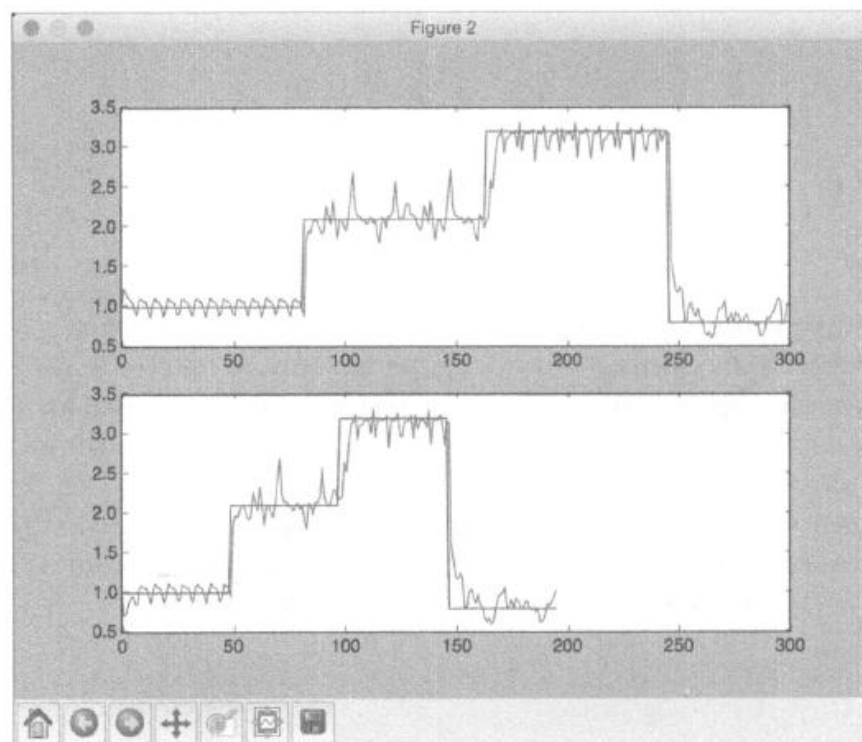
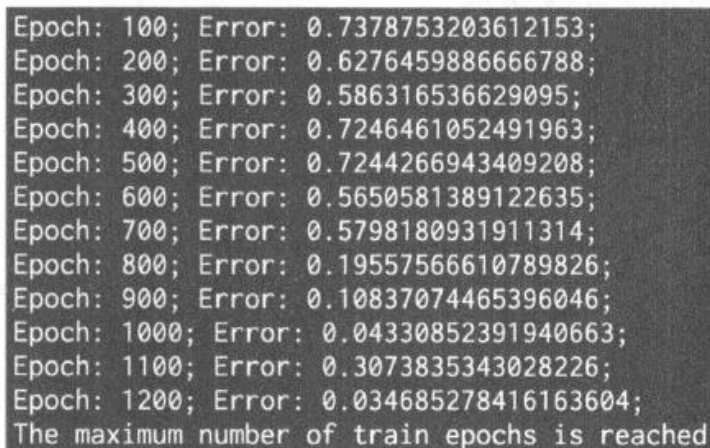


Рис. 14.13

В окне терминала отобразится следующая информация (рис. 14.14).



```
Epoch: 100; Error: 0.7378753203612153;  
Epoch: 200; Error: 0.6276459886666788;  
Epoch: 300; Error: 0.586316536629095;  
Epoch: 400; Error: 0.7246461052491963;  
Epoch: 500; Error: 0.7244266943409208;  
Epoch: 600; Error: 0.5650581389122635;  
Epoch: 700; Error: 0.5798180931911314;  
Epoch: 800; Error: 0.19557566610789826;  
Epoch: 900; Error: 0.10837074465396046;  
Epoch: 1000; Error: 0.04330852391940663;  
Epoch: 1100; Error: 0.3073835343028226;  
Epoch: 1200; Error: 0.034685278416163604;  
The maximum number of train epochs is reached
```

Рис. 14.14

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

1. Какие нейронные сети называют рекуррентными?
2. В чем особенность рекуррентных нейронных сетей?
3. Чем отличается рекуррентная нейронная сеть от многослойного перцептрона?
4. Привести код на Python рекуррентной нейронной сети для анализа последовательных данных, рассмотренной в методических указаниях.
5. Самостоятельно провести эксперименты с рекуррентной нейронной сетью, сгенерировав сигналы произвольных синусоидальных функций. Представить графики продвижения процесса обучения, а также предсказанный результат, наложенный на суммарный входной сигнал.

```

import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl

def get_data(num_points):
    # Create sine waveforms
    wave_1 = 0.5 * np.sin(np.arange(0, num_points))
    wave_2 = 3.6 * np.sin(np.arange(0, num_points))
    wave_3 = 1.1 * np.sin(np.arange(0, num_points))
    wave_4 = 4.7 * np.sin(np.arange(0, num_points))

    # Create varying amplitudes
    amp_1 = np.ones(num_points)
    amp_2 = 2.1 + np.zeros(num_points)
    amp_3 = 3.2 * np.ones(num_points)
    amp_4 = 0.8 + np.zeros(num_points)

    wave = np.array([wave_1, wave_2, wave_3, wave_4]).reshape(num_points * 4, 1)
    amp = np.array([amp_1, amp_2, amp_3, amp_4]).reshape(num_points * 4, 1)

    return wave, amp

# Visualize the output
def visualize_output(nn, num_points_test):
    wave, amp = get_data(num_points_test)
    output = nn.sim(wave)
    plt.plot(amp.reshape(num_points_test * 4))
    plt.plot(output.reshape(num_points_test * 4))

if __name__ == '__main__':
    # Create some sample data
    num_points = 40
    wave, amp = get_data(num_points)

    # Create a recurrent neural network with 2 layers
    nn = nl.net.newelm([[ -2, 2]], [10, 1], [nl.trans.TanSig(), nl.trans.PureLin()])

    # Set the init functions for each layer
    nn.layers[0].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
    nn.layers[1].initf = nl.init.InitRand([-0.1, 0.1], 'wb')
    nn.init()

    # Train the recurrent neural network
    error_progress = nn.train(wave, amp, epochs=1200, show=100, goal=0.01)

    # Run the training data through the network
    output = nn.sim(wave)

    # Plot the results
    plt.subplot(211)
    plt.plot(error_progress)
    plt.xlabel('Number of epochs')
    plt.ylabel('Error (MSE)')

    plt.subplot(212)
    plt.plot(amp.reshape(num_points * 4))
    plt.plot(output.reshape(num_points * 4))
    plt.legend(['Original', 'Predicted'])

    # Testing the network performance on unknown data
    plt.figure()

    plt.subplot(211)
    visualize_output(nn, 82)
    plt.xlim([0, 300])

    plt.subplot(212)
    visualize_output(nn, 49)
    plt.xlim([0, 300])

    plt.show()

```

Рис. 14.П1. Пример кода рекуррентной нейронной сети для анализа последовательных данных на Python

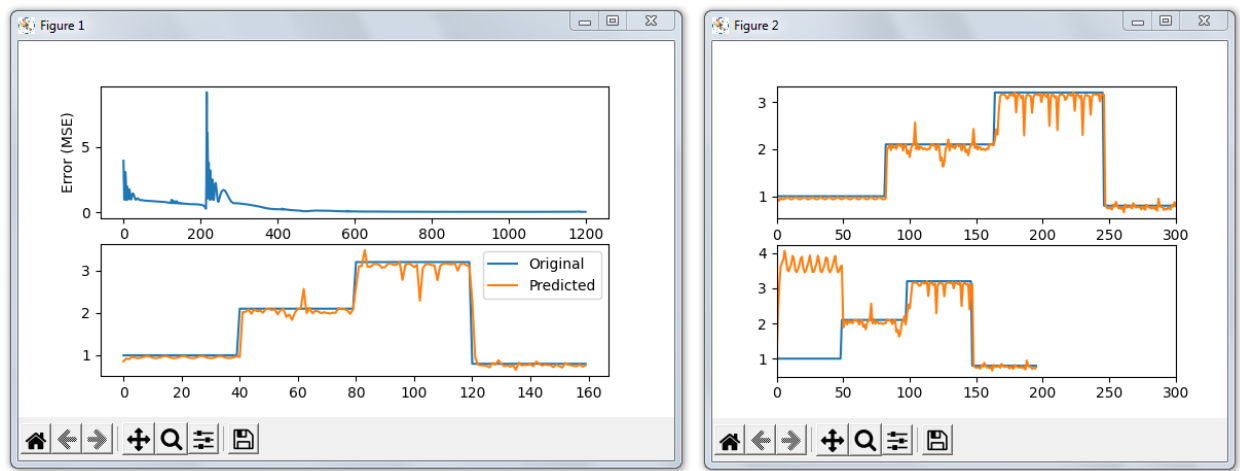


Рис. 14.П2. Продвижение процесса обучения и график предсказанных результатов, наложенный на суммарный входной сигнал

```
Epoch: 100; Error: 0.8172968161193781;
Epoch: 200; Error: 0.5882877063773864;
Epoch: 300; Error: 0.688625630970157;
Epoch: 400; Error: 0.23772679095539706;
Epoch: 500; Error: 0.13110752628953598;
Epoch: 600; Error: 0.06879942296592026;
Epoch: 700; Error: 0.04527349101335909;
Epoch: 800; Error: 0.03974127156822413;
Epoch: 900; Error: 0.03772552049799407;
Epoch: 1000; Error: 0.037138757455700465;
Epoch: 1100; Error: 0.03571109555483526;
Epoch: 1200; Error: 0.03855738748322542;
The maximum number of train epochs is reached
```

Рис. 14.П3. Результат выполнения кода